pShare and pLogger

MOOS sharing and logging utilities.

Included MIT MOOS-IvP PDF sources:

- app_pshare.pdf
- app_plogger.pdf

Sharing Data with pShare

The `pShare` application was written by Paul Newman.

http://www.robots.ox.ac.uk/~pnewman/MOOSV10-docs/examples-docs-moos/pShare/pShare.pdf

1 Logging Data with pLogger

The pLogger process is intended to record the activities of a MOOS session. It can be configured to record a fraction of or every publication of any number of MOOS variables. It is an essential MOOS tool and is worth its weight in gold in terms of post-mission analysis, data gathering and post-mission replay.

The configuration of pLogger is trivial and consists of multiple lines with the following syntax:

```
Log =  varname @ period [NOSYNC],[MONITOR]
```

where varname is any MOOS variable name and period is the minimum interval between log entries that will be recorded for the given variable. For example if varname = INS_YAW and period = 0.2 then even if the variable is published at 20Hz it will only be recorded at 5Hz. The optional NOSYNC flag indicates that this variable should not be recorded in the synchronous logs (see section 3). The optional MONITOR flag tells pLogger to send a notification if this variable isn't logged at least every 10 seconds. The notification occurs under the MOOS_DEBUG variable. If you are running iRemote (which subscribes to this variable automatically) you'll see a warning printed to the screen. This can be pretty useful when runnning a complicated system and you really do want notification that an important variable isn't being logged (probably because the process producing the variable is kaput in some sense.)

2 Logging Session

The logger supports the notion of Logging Sessions. For each log session the Logger will create a new directory and place all the logged files within that directory. These are typically (see later sections) an alog file, an slog file, a system log file, a copy of the mission file (moos file), and a copy of the behavior file (bhv file). A new log session can be created by writing the variable LOGGER_RESTART to the MOOSDB or if you are using iRemote by pressing shift-g.

3 Log File Types

The logger records data in two file formats - synchronous (”slog” extensions) and asynchronous (”alog” extensions). Both formats are ASCII text – they can always be compressed later and usability is more important than disk space. The two formats are now discussed.

3.1 Synchronous Log Files

Synchronous logging makes a table of *numerical* data. Each line in the file corresponds to a single time interval. Each column of the table represents the broad evolution of a given variable over time. The time between lines (and whether synchronous logging is even required) is specified with the line

```
SyncLog = true/false @ period
```

where *period* is the interval time. If there has been no change in the numeric variable between successive time steps then its value is written as NaN. It is important to note that synchronous logs do not capture all that happens - they sample it. Synchronous logs are designed to be used to swiftly appraise the behaviour of a MOOS community by examining numeric data in a tool such as Matlab or a spreadsheet. The MOOSData Matlab script reads in these files and with a single mouse click can display the time evolution of any logged variable.

3.2 Asynchronous Log Files

Asynchronous logging is thorough. The mechanism is designed to be able to record *every* delta to the MOOSDB. The use of the period variable allows the mission designer to back off from this ultimate limit and record variables at a maximum frequency. The key properties of asynchronous can be enumerated as follows:

1. Records both string and numeric data
2. Records data in a list format - one notification per line
3. Entries only made when variable is written

Asynchronous log files are designed to be used with a playback tool (for example uPlayback or other purpose-built executable). Although the handling of strings and numeric data adds a slight overhead to such a program's complexity the utility gain from being able to slow, stop and accelerate time during a post-mission replay/reprocessing session is simply massive. Turn asynchronous logging on with:

```
AsyncLog = true
```

in the logger configuration block.

4 System Log File

There is a third kind of log file that is produced - a system log file (ylog). This file only contains data contained in MOOS_SYSTEM and MOOS_DEBUG messages. The later can be written to by calling the CMOOSApp::MOOSDebugWrite method from any CMOOSApp derived class. The thinking behind the ylog files is that its pretty useful to be able to browse through a text file of events to see when and if things went wrong in a mission. Processes can write to MOOS_SYSTEM and or MOOS_DEBUG if something happens which they think is salient. Think of this as /var/log.

5 Dynamic Logging Configuration

Post V7.0.1 releases of MOOS include a dynamic logging ability. External parties can, at run time request the logger to start logging particular (named) variables. The logger subscribes to messages called PLOGGER_CMD (if the Logger is being run under the MOOS name "pLogger" otherwise subsitute the relevant name) and by correctly formatting the string data of this message dynamic logging can be invoked.

```
LOG_REQUEST =  varname @ period [NOSYNC],[MONITOR]
```

Note that this string format is identical to that found in the mission file modulo `LOG` being replaced with `LOG_REQUEST`. Now dynamic logging sits naturally with ”alog” (asynchronous) files but not so well with ”slog” files. These files are easy to use(and hence popular) because they are rectangular numerical array written to text file (easily parsed by Matlab’s load command). But dynamic logging means that by the time a mission ends an unknown number of variables (columns) will occur in every line of the slog. To address this issue the logger can be configured (in its mission file configuration block) with a line like

```
DynamicSyncLogColumns =  30
```

Here 30 columns are being reserved for variables that are requested at run time. As dynamic requests are received and processed by the logger these unclaimed columns are consumed until none remain. At this point any future dynamic logging requests will still be accepted but the logged variables won’t appear in the slog file (they will of course appear in the alog file). Unclaimed slots will be labelled `DYNAMIC_X` until claimed and will always have `NaN` entries.

6 Specifying Log File Names and Locations

Each time the logger starts creates (if required) a new directory in the logging root directory (see below) and performs logging within that directory. `pLogger` is quite flexible in terms of log file configuration and is controlled by the following variables.

- `GlobalLogPath`: This is a **file scope variable** (ie not in any process configuration block). If it is present in a mission file then it specifies the root directory in which log files will be created.
- `Path`: specifies the root logging directory but only used if `GlobalLogPath` is not set (see above). This path must exist, `pLogger` will not create it on startup. If the path does not exist, `pLogger` will default to the current working directory (‘.’) as described below.
- `File`: this is the stem file name given to logged files (alog/slog and ylog)
- `FileTimeStamp`: if this is set to true, the name of each logged file (and created containing directory) will be the concatenation of the `File` variable and a time stamp. If `FileTimeStamp` is false then each time the logger is run it will write to the same set of log files **and destroy the original contents**. This is by design, when developing it’s often useful to not have useless log files take ever more space on your machine.

Note that if the logger is run without a mission file it starts logging to the local directory with file time stamping enabled. See Listing 1 for a typical logger configuration block.

If for some reason the logger was unable to start logging to a location specified in the log file it trys, as a last resort, to open a log directory in the current working directory (‘.’). If this fallback fails then all is lost, we are without hope and the Logger exits. If you are running a mission from pAntler you’ll be notified that the logger has quit.

7 Mission Backup

Simply having the *alog* and *slog* files is not enough to evaluate the mission. One also needs the things that *caused* the data to be recorded, namely the `*.moos` mission file or the `*.bhv` behavior file.

To this end the pLogger process takes a copy of these files and places them (name appended with a time stamp if desired) within the logging directory. The files extensions are renamed to `*._moos` and `*._bhv` respectively.

7.1 Additional File Backup on Demand

It is quite within the bounds of reason that a system using MOOS makes use of configuration (or other files) which has content which should not reside inside a *.moos or *.hoof file. pLogger provides a mechanism to back these files up on request at run time. Post V7.0.1, by writing to the PLOGGER_CMD variable any MOOS process can instruct the logger to back a local file up and place a copy in the current logging directory. See 6. As always the process _CMD message is a string and for dynamic file back up should be formatted as:

```
COPY_FILE_REQUEST = PathToFile
```

for example

```
COPY_FILE_REQUEST = /home/VehicleQ/CameraCalibration.txt
```

would log `"/home/VehicleQ/CameraCalibration.txt"` to a file called `"CameraCalibration._txt"` in the current logging directory. Note the additional underscore which is in keeping with the backup style used for mission files. If the file being backed up has no extension a `"._bak"` is added.

7.2 Wildcard Logging

Post V7.0.1 a new configuration option has been introduced which instructs the logger to log every change made to the MOOSDB to an alog file. (It does this by using a specialised MOOS IPC call to the DB which is not intended for daily use). By writing

```
WildCardLogging =  true
```

Asynchronous logging will be turned on (even if `AsyncLog = false` is present in the configuration block) and every change to every variable will be logged to the current alog file. Some points to note:

- Wildcard logging only causes variables to be written to the current alog file. Synchronous logging is not affected
- If the log rates for variables have already been been specified at configuration then it is these rates that are used
- If wild card logging is enabled, when the logger spots that variables that are not already known to it are being written to the MOOSDB it registers to be told about every change made (like `Log = XXX @ 0`).
- By default wildcard logging is disabled. It was designed to be a safety net for situations in which many new variables are being created by a software team in a dynamic environment.
- It may take up to a second for the logger to detect and subscribe for new MOOS variables. If

new data is written multiple times before subscription can complete (i.e within a second), the logger will be unable to capture these writes.

8 Runtime File Backup

By writing a correctly formatted string value to the MOOS variable PLOGGER_CMD, processes within the MOOS community can request the logger to back up arbitrary files to the current log directory. For example if the contents of a message with name PLOGGER_CMD are set to

```
PLOGGER_CMD = "COPY_FILE_REQUEST=/home/pnewman/code/TheFile.xzy"
```

and published to the MOOSDB it will result in a copy of `/home/pnewman/code/TheFile.xzy` being placed in the logging directory under the name `TheFile._xzy` - note the additional underscore. If the file in question has no extension then `._bak` is added appended to the file name.

9 Example Configuration

Listing 9.1: An example pLogger configuration block.

```
//-----------------------------------------------------
// pLogger configuration block
ProcessConfig = pLogger
{
    //over loading basic params..lets be fiesty
    AppTick  = 20.0
    CommsTick   = 20.0

    //all file names begin with this stem
    File        = SciPark29Mar

    //where is the root log directory
    PATH        = /home/doe/MOOSData/SciencePark/

    //yes we want some sync logging for crude
    //performance checking
    SyncLog     = true @ 0.2

    //yes we want async logging so we can replay
    // exactly what happened and record strings
    AsyncLog    = true
```

```

    WildCardLogging = false

    //yes append each created directory log file
    //with a time stamp DAY MONTH YEAR TIME
    FileTimeStamp   = true

    //what do we want to log (zero means capture everything)
    Log     = LMS_LASER_2D   @ 0 MONITOR
    Log     = LMS_LASER_3D   @ 0 MONITOR
    Log     = MARGE_ODOMETRY @ 0
    Log     = DESIRED_RUDDER @ 0
    Log     = DESIRED_THRUST @ 0
    Log     = CAMERA_GRAB    @ 0
    Log     = GPSData        @ 0.4
}
```